REVON: LOG AND MERKLE DIFFERENCING FOR DURABLE STRUCTURED DATA VERSIONING

BY ATHARVA SHEERSH PANDEY[1,a], ADHYAN JAIN[1,b], AND POORNIMA NEDUNCHEZHIAN[1,c]

1 School of Computer Science and Engineering, VIT Vellore

a atharva.sheersh2024@vitstudent.ac.in; b adhyan.jain2024@vitstudent.ac.in

c Corresponding author: Poornima Nedunchezhian (poornima.n@vit.ac.in)

Revon is a Python prototype for storing durable, linear key/value histories in a SQLite-backed Merkle hash trie. We use it to compare log aggregation with hash-pruned differencing over one history, assess an operation-count selection rule, and measure complete versioning workflows. On Linux, 516 executions are accompanied by a separate 72-execution geometry and telemetry supplement. Eight of nine paired hybrid/Merkle timing intervals favour hybrid; the dense interval includes parity. A separate threshold control exposes a slower choice: the rule selects Merkle when log is faster. The Windows evaluation uses 93 cases, each in a fresh worker, with disjoint calibration and validation histories. It covers controlled sensitivity experiments, Wine Quality and CO2 sources, and repeated runs at 100,000 and 1,000,000 records. The calibration winner uses log across every tested interval, so these results leave the benefit of switching unresolved. The CO2 histories contain observed publication changes; the wine corrections are generated. We verify diffs and historical states with semantic oracles and keep algorithm, CLI-phase and whole-worker measurements distinct. The findings apply to the recorded histories and machines. They do not establish an optimal crossover, storage-engine superiority or reproduction by an independent team.

1. Introduction. An application that retains dataset versions needs to recover earlier states and find the records that changed between them. Saving a complete file for every version addresses part of that need, but the application must still maintain and compare record-level history. Git provides a familiar approach to file history: content identifies immutable objects, and parent commits connect them [10]. Those objects describe files and directories. Record histories raise a further concern. The keys may stay the same across versions even while the density of updates and the number of intervening commits vary.

We keep the durable history fixed and compare two routes to its endpoint changes. One route composes the operations in a log; the other compares Merkle hashes and skips subtrees that match. Both return the same semantic diff from the same representation. To make that comparison possible, Revon integrates established path copying and content addressing with SQLite transactions. We also evaluate a rule that chooses between the routes. Our timings cover the versioning operations and workloads measured in this study, rather than structured-data management as a whole.

Existing systems provide the context for this narrower comparison. Dolt combines SQL with a commit graph and Prolly Trees [13, 28], ForkBase offers POS-Tree/SIRI storage with fork semantics [32], and Noms describes history-independent chunked collections [3]. Revon exposes key/value operations over a linear history so that diff paths can be

Keywords and phrases: data versioning, Merkle hash trie, hybrid differencing, content-addressed storage, benchmark methodology.

compared under controlled conditions. This scope does not imply parity with those systems' features or introduce a new family of persistent indexes. For cross-system measurements we invoke Revon through its in-process API and Dolt through its CLI, recording separate CLI phases to make the boundary of the comparison explicit.

1.1. Research Questions. RQ1: How do latency, persistent storage and worker resources trade off in complete versioning workflows, and what does the API/CLI boundary mean for interpreting those measurements?

RQ2: How well does selection by operation count perform on generated histories withheld from calibration, relative to forced log and forced Merkle over the same representation?

RQ3: Under controlled changes to the tested histories, what effects do workload structure, payload size and fixed-trie geometry have on differencing cost?

1.2. Contributions. The implementation combines atomic SQLite persistence, content addressing and path copying in a prototype with two diff paths. Both paths follow one encoded-value contract. This integration, and the ability to compare the paths under matched conditions, form the engineering contribution.

For path selection, we define a rule based on operation count and compare its decisions with forced execution of each path. We finish calibration and record the threshold before testing held-out histories. Cases in which selection is slower remain in the analysis, which also asks whether choosing log throughout would be enough for these histories.

The verification evidence comes from checking observed diffs and historical states against oracles built independently of the storage implementation. Executed sources, raw measurements, environment records and data-preparation procedures are retained with the paper. The materials also document the correctness checks and what the telemetry can and cannot measure.

We report Linux workflow experiments alongside a 93-case Windows evaluation. In the Windows design, validation seeds are separate from calibration seeds; matched sensitivity controls, Wine Quality and CO2 sources, and repeated scale runs extend the comparison. The campaigns use different experimental units, so each is analysed on its own and their timing rows are kept separate.

The evidence is bounded by a single-writer store, linear histories and the workloads tested. It can show trade-offs within those conditions. It cannot identify a universally best selector or geometry, establish query-engine superiority, or demonstrate reproduction by an independent team.

2. Related Work.

2.1. Content Addressing and Persistent Structures. A collection represented by a Merkle root can be authenticated through that root [22]. Content addressing serves software histories in Git and networked content in IPFS [10, 5]. Persistent structures use path copying to retain old states while sharing the parts that have not changed [14]. Ordered stratified B-trees [31] and fully persistent B-trees [8] address disk-based dictionaries. Revon uses a different routing choice: hashed keys pass through a fixed-depth trie, where matching hashes can stop a comparison but do not provide ordered range access. SciDB and TimeArr handle versioned arrays with layouts and deltas suited to arrays [27, 30]. TardisDB supports version access in main memory, and Saquib et al. examine replicated versioned structures in wide-area systems [25, 26]. These designs help explain the choices

relevant here, although they do not compare Revon's log and Merkle paths on a common history.

Merkle Search Trees reconcile states by combining search-tree ordering with Merkle authentication [4]. Revon also prunes comparisons by hash, but uses fixed-depth routing and centralized linear history instead of a replicated CRDT. LBFS finds redundant content through content-defined chunking [23]; related chunk-boundary methods appear in Dolt and Noms. Such boundaries can preserve deduplication as data moves, and ordered indexes permit range operations. Neither of those capabilities is measured in the Revon benchmark.

2.2. Structured Dataset Versioning. A dataset-versioning system must support collaborative work as well as storing and reconstructing states. DataHub investigates collaborative histories at scale [6]. Decibel supports relational branching through version-first, tuple-first and hybrid layouts [20]. OrpheusDB uses partitioning to improve version retrieval when an existing DBMS acquires versioning support [19]. The storage/reconstruction trade-off is formalized by Bhattacherjee et al. [7]. Delta Lake supplies ACID tables through transaction logs and checkpoints [2], and version-graph methods select materialized states and deltas under constraints on storage and retrieval [18]. Consistent release identification and description are the concern of the FAIR-compliant proposal [16], which complements this storage work without specifying a diff algorithm. Revon's key/value interface addresses only part of that broader problem.

2.3. System Design Gap. Noms stores structured values in a Merkle DAG and provides diff and synchronization [3]. ForkBase combines content addressing with fork and deduplication semantics [32]. In Dolt, content-addressed Prolly Trees underpin structural sharing, native diff and ordered indexing [13, 28]. Git4Data exposes snapshot, branch, diff and merge operations through SQL on a relational engine [17]. Its available arXiv preprint is cited for that abstraction; we do not treat its reported results as peer-reviewed or independently reproduced evidence. Revon has a smaller implementation scope: linear history and fixed-depth hash routing in a single-writer key/value store, without SQL or a branching interface.

The systems and studies most relevant to these implementation decisions are collected in Table 1.

Table 1
Related systems and studies

System or study	Representation or focus	Relevance to Revon
Persistent DS [14]	Path copying	Formal structural-sharing basis
Git [10]	Object DAG	Identity and commit ancestry; file-oriented
IPFS [5]	Merkle DAG	General content-addressed versioning
MST [4]	Merkle search tree	Authenticated ordered reconciliation
DataHub [6]	Version graph	Collaborative dataset motivation
Decibel [20]	Relational layouts	Native database branching
OrpheusDB [19]	Partitioned versions	Storage/retrieval optimization
Delta Lake [2]	Log/checkpoints	ACID time travel at data-lake scale
Noms [3]	Merkle DAG	Structured content addressing
ForkBase [32]	Forkable CAS	Fork and deduplication semantics
DoltHub [13, 28]	Prolly Trees	Mature production comparator

2.4. Design Implications and Research Positioning. Representation choices move work between version creation, retention, reconstruction and comparison. A complete snapshot is easy to retrieve but copies unchanged records. A log leaves work for replay or checkpointing. With path copying, the unchanged structure is shared and only update-affected paths are rebuilt [14, 7]. Decibel and OrpheusDB show how layout affects version creation and retrieval [20, 19], and version-graph analyses make the storage/retrieval trade-off explicit [18]. Reproducibility research considers how builds and artifacts can be kept available for inspection [1]. We therefore report import, commit, checkout, diff and storage separately. Retaining an artifact enables inspection; it is not evidence that the implementation runs faster.

Different studies also mean different things by a change. Differential dataflow updates query results when timestamped inputs change [21]. Provenance describes the origins of output facts and why those facts occur [9]. DSLog captures fine-grained array lineage and supports queries over it [33]. Explain-Da-V relates semantic dataset changes to transformations [29], while Baseline uses operation-based differencing for refactorings and schema changes [15]. Revon restricts its output to keys added, removed or modified between two states. Causes, lineage, effects on queries and schema intent are not inferred. The related studies provide context, but their workloads and outputs are too different to support direct timing comparisons with these key/value diffs.

Each Revon-H commit retains both a content-addressed changeset and a Merkle root, allowing either path to recover the same endpoint changes. The selection rule changes how the diff is computed. We expect log aggregation to suit short, sparse intervals and hypothesize that hash pruning may help after longer sequences of operations; whether their costs cross has to be measured. Git4Data shows the wider interest in database-native version control [17]. For our fixed-depth-trie comparison, Snapshot and Log-only serve as representation baselines. The public workloads consist of generated TLC and wine corrections and observed changes in a CO2 publication sequence. Their results do not

establish performance across independent hardware, concurrent workloads or production systems.

2.5. Capabilities and comparison boundaries. Table 2 distinguishes what the systems document from the evidence used in this paper. Feature descriptions are not benchmark results. ForkBase and Noms describe typed-collection APIs, but we evaluate no relational layer over either. The Git4Data preprint describes a different abstraction level [17]. A capability left unevaluated here should not be interpreted as absent from the system.

Table 2
System capabilities and evidence scope; documented features are not performance results

System	Representation / history	Diff / persistence	Branches / queries	Reproducibility and use here
Revon	Fixed-depth hash trie; linear history	Log or Merkle; SQLite objects	No named branches/merge; key/value API	Executed snapshots, semantic proofs and raw trials; measured here
Dolt [13, 28]	Prolly Trees; commit graph	Native version diff; persistent block store	Branches/merge; SQL	Official docs/source; CLI workflow measured, engine not isolated
ForkBase [32]	POS-Tree/SIRI; forkable versions	Hash-pruned diff/merge; addressed chunks	Fork on demand/conflict; typed KV APIs	Primary paper and APIs; no experiment run here
Noms [3]	History-independent chunks; Prolly collections	Hash-based diff/sync/merge; chunk store	Typed collection API; not benchmarked here	Technical overview/source; conceptual comparison
Git4Data [17]	MatrixOne immutable objects/MVCC; table versions	Database-native diff/merge	Snapshot/tag/branch ; SQL extensions	Preprint/BranchBench claims belong to its authors; not run here

2.6. Related foundations and scope boundaries.

Research-data versioning guidance includes version identity and descriptions of change as requirements for reliable citation and reuse [34]. The IPFS evaluation and FastVer examine content-addressed persistence and integrity beyond the setting of our local prototype [35, 36]. Prior states can also be retained and verified with persistent tries, B+-trees, external-memory search trees, snapshot stores and authenticated data structures [37-41]. Repository views, collaborative analytics and DEX illustrate how history systems support access to repositories and collaborative work [42-44].

Schema-evolution and temporal-database research addresses changes in the structure and meaning of data over time [45-47]. Work on provenance traces where results come from and how they are derived, using semirings and accounts of scientific workflows as well as tutorials and broader syntheses [48-51]. Further studies cover aggregate-query provenance, authenticated or secure provenance, FAIR stewardship, privacy and the PROV model [52-56]. In SEEK, provenance and data management are brought together within a scientific repository [57].

Several storage designs offer context for our tree and log mechanisms. Foundational B-tree work and cache-oblivious indexing explore different search and update costs [58-61]. LSM trees, Bigtable, Dynamo and log-structured file systems are examples of log- or append-oriented storage [62-65]. WiscKey and LogBase study write-efficient key-value and log-

based database storage, and multi-version partitioned B-trees handle version-aware indexing [66-68]. They motivate comparisons of design choices, but the workloads and measurements in those studies cannot replace the ones evaluated here.

3. Revon Design. Figure 1 shows the implemented architecture and places the evaluated operations within it.

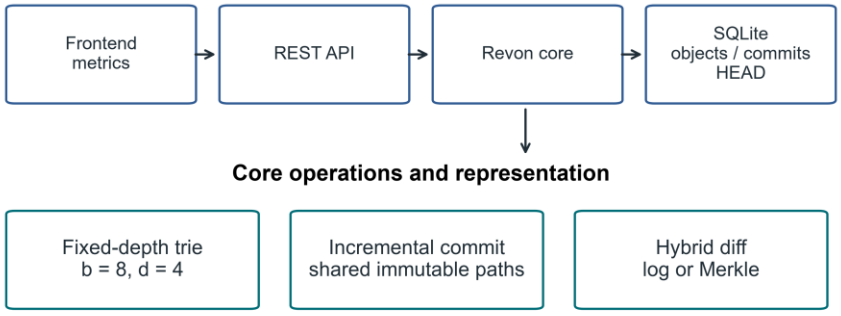


Fig 1. Revon architecture showing the implemented scope of the research prototype

3.1. Addressed Object Model. Revon names a trie node or changeset by the SHA-256 digest of its canonical JSON serialization. The digest for a commit includes the root, parent, changeset, message and timestamp. Human-readable names such as v1 and v2 are aliases and do not affect object identity. The current commit is identified by a durable HEAD reference.

3.2. Fixed-Depth Merkle Hash Trie. For the main configuration, b=8 and d=4. Each routing level consumes three bits of SHA-256(key), giving twelve bits that select one of 4,096 possible leaf buckets. Leaves retain complete keys, so keys sharing a routing prefix remain distinguishable. Expected occupancy under uniform hashing is N/b^d. Other b,d combinations are evaluated separately; we do not assume the default minimizes cost.

3.3. Incremental Hashing and Commit. A batch is processed by canonicalizing its values and grouping keys with the same routing prefix. Revon rewrites each touched leaf once, then rebuilds the ancestors on the affected paths. Unchanged child hashes remain in internal nodes, sharing the structure beneath them. The objects, changeset, commit metadata, version alias and measurements are written and HEAD is updated in a single SQLite transaction. Reopening the repository checks stored payload hashes and references.

3.4. Hybrid Diff. Log differencing composes operations by key while traversing the requested ancestor interval. Merkle differencing compares roots recursively, stops when hashes match and examines entries beneath unequal paths. Revon-H validates the version identifiers, checks ancestry in both directions and counts changeset operations before selecting a path. Log is used for an eligible ancestor interval whose count is no greater than T; otherwise selection goes to Merkle. A reverse log request exchanges old and new values, while a request comparing a version with itself returns an empty diff. Forced log rejects unrelated versions. Unknown identifiers raise an error. Revon-M always requests Merkle, with ancestry still checked by its public version-pair API. Both paths produce the same sorted semantic diff, and commit and checkout are timed independently.

The Linux workflow protocol sets T=4,096. For a separate Linux control, an auxiliary calibration experiment selected T=16,384. T measures operation count, whereas b^d determines bucket capacity. Neither parameter was fitted to the Linux results.

3.5. JSON value ownership and identity. Revon accepts JSON nulls, booleans, finite numbers, strings, lists and objects with string keys. Input containers are validated and detached before storage, and containers returned through reads or diagnostic views are detached too. Immutable scalars may be shared. Before writing, Revon rejects unsupported Python values, nested non-string keys, non-finite numbers and cyclic containers. Its v1 JSON encoding defines equality through sorted object keys, compact separators, UTF-8 and the exclusion of NaN. The encodings of true, 1, 1.0 and signed floating zero remain distinct. These rules specify the Python prototype's encoding contract, without claiming conformance to a cross-language canonical-JSON standard.

Algorithm 1 (batch update). Begin by validating the batch and taking ownership of the values. Before any object is interned, reject keys that occur in both puts and deletes and requests to delete keys that are missing. Read the old values and retain mutations only where encoded identity changes. Group those mutations by routing prefix, rewrite each affected leaf, sort the entries and hash the complete payload. Rebuild the affected ancestors using unchanged child hashes where possible. Record the content-addressed changeset and commit. Detaching values prevents callers from altering stored history through either their input containers or supported reads.

Algorithm 2 (log composition). Walk the ancestor interval in chronological order. For every key, retain its existence and value at the first old endpoint and the last new endpoint. Keep it in the result unless the two endpoints agree in both existence and encoded value. Insertion followed by deletion cancels, as does an update that is later restored. Null is still different from an absent key. Sort the keys left in the result; for reverse requests, exchange endpoints and swap the added/deleted labels.

Algorithm 3 (selection). Validate the version pair and check whether either version is an ancestor of the other. Add the changeset operation counts for the interval. Choose log if the interval is eligible and the total is at most T; otherwise choose Merkle. Merkle traversal bypasses equal-hash subtrees and compares values in unequal leaves. Apply the same encoded-identity rule on both paths, detaching values returned to the caller.

Cost model. Let N be the record count, d the depth and b the branching factor. Write m for the keys in a batch, h for interval length, p for accumulated operations, u for distinct keys in the interval and s for serialized output bytes. Expected leaf occupancy is N/(b^d) with uniform routing, but can reach N in the worst case. Besides validating and copying inputs, a batch performs m routed old-value lookups and scans occupied leaves. It sorts and serializes touched leaves and copies affected internal child maps. This work prevents a dataset-independent O(d) bound for batch processing.

Log composition traverses ancestry with O(p + u log u) work, plus the costs of encoded-value comparison and output copying/serialization. Merkle cost varies with visited node pairs, child-slot sorting and examined leaf entries; value comparison and output sorting add to it. Both public version-pair modes traverse ancestry. When a request is reversed, the unsuccessful direction may first walk back to genesis. Constructing output requires work at least proportional to s. Large JSON values incur additional comparison and copying that scalar-only benchmarks cannot characterize.

Retained nodes, changesets and commits are eagerly loaded from SQLite. Although durable on disk, they stay resident while Revon runs, so the retained history must fit in memory. Retained payload also affects repository opening and verification. The prototype implements no disk-page cache or eviction policy. It uses SQLite transaction atomicity for persistence; production concurrency and operational guarantees would require further evaluation.

Figure 2 traces the selection process for a version pair, including eligibility checks and reverse requests.

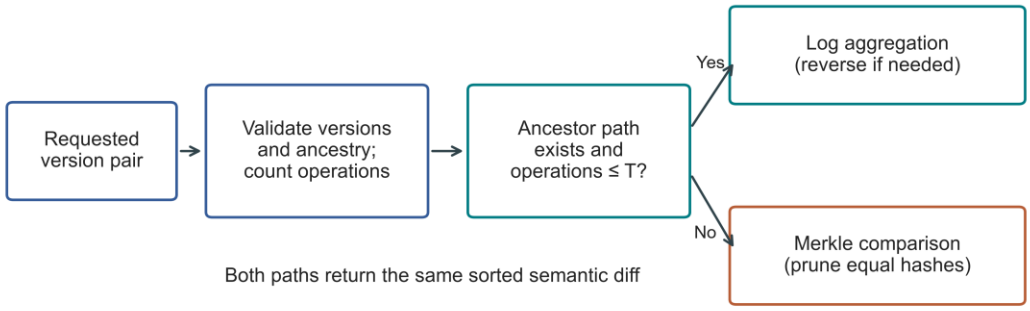


Fig 2. Diff-path selection for a version pair. T gives the operation threshold. Invalid versions raise an error before a path is chosen, and both paths use the same persistent trie

4. Experimental Methodology.

4.1. Environment and evidence boundaries. The Linux experiments ran on an Ubuntu n2-standard-4 cloud VM, whose environment and executed sources appear in Table 3. Windows experiments ran on a separate computer operated by the authors. The authors conducted both campaigns. Since hardware, OS, runtime and storage change together between them, differences cannot be attributed to any one factor.

Table 3
Linux execution environment and the recorded sources

Property	Recorded configuration
Machine	GCP n2-standard-4; Intel Cascade Lake; 4 vCPUs; 16 GB RAM; us-east1-c
Operating system	Ubuntu 24.04.5 LTS; x86_64; kernel 7.0.0-1011-gcp
Storage	100 GB balanced persistent disk; no swap configured
Runtime	CPython 3.14.3, Clang 22.1.1 build; GIL enabled; JIT not enabled; SQLite 3.50.4
Comparator and packages	Dolt 2.3.1; psutil 7.1.0; PyArrow 25.0.1
Primary source	linux-validation-20261008: executed source, fixed before measurement
Supplement source	linux-final-supplement-20261008: same core; zero-aware I/O sampler and geometry history checks
Background conditions	Google Ops Agent active; filesystem cache not cleared; monitoring included

The campaign records include executed-source hashes, a fixed protocol, environment information, correctness checks and raw observations. Linux preflight suites passed 67 and 69 tests; the Windows suite passed 77. Setup failures and the public-data exclusion applied before timing remain documented. Appendices A and B give access information and detailed procedures.

4.2. Variants and workloads. We measure six workflows across five systems. Snapshot stores a complete state at each version, whereas Log-only stores an initial state followed by operations. Revon-M requests Merkle diff; Revon-H chooses the path. Dolt stores records in a primary-key SQL table and produces its native JSON diff. SQL INSERT and CSV table import are timed as separate Dolt workflows. Subsequent operations conform to a shared semantic-output contract despite differences in interfaces and storage features. The figure workload IDs are defined in Table 4.

Table 4
Linux workloads, each primary history having ten generated commits

ID / scenario	Rows	Changes / commit	Locality
S1 Small sparse	1,000	10	Spread
S2 Medium sparse	10,000	10	Spread
S3 Medium dense	10,000	1,000	Spread
S4 Large sparse	100,000	100	Spread
S5 Application-key local	100,000	100	Lexically adjacent keys
S6 Range local	100,000	100	Contiguous key intervals
S7 Repeated key	100,000	100	Repeated small key set
S8 Hash-route local	100,000	100	Shared 9-bit hash prefix
P1 TLC public records	100,000	100	Generated fare corrections

The eight synthetic scenarios account for 432 executions: each has six variants with two warm-ups and seven measured trials. Public-record workloads contribute 27 executions for Revon-M, Revon-H and Dolt SQL. Threshold controls contribute 54, and the three million-row diagnostics bring the total to 516. This comprises 114 warm-ups, 399 measured trials and three diagnostics. All executions finished with verified output. No failed timing execution is silently replaced or excluded from the accounting.

4.3. Public dataset and generated histories. TLC input comes from the January 2024 yellow-taxi release [24]. A checksum fixes the Parquet input at 2,964,624 rows. Using seed 20261001, a dispersed affine permutation produces nested subsets of 10,000, 100,000 and 1,000,000 rows. The eleven retained fields are vendor, pickup/drop-off times, passenger count, distance, pickup/drop-off locations, payment type, fare, tip and total. Keys comprise the release, file-checksum prefix and physical row ordinal. They identify rows in that file and do not assert trip identity across releases. Values, including nulls, are serialized as compact UTF-8 JSON with sorted keys. Every row serializes successfully, including the 47,277 missing passenger counts in the million-row subset. Identical field values do not cause rows to merge. This affine procedure does not sample uniformly at random.

We generate ten commits for each TLC history. In the Linux 100,000-row workload, a commit changes 100 records; in the million-row diagnostic it changes 1,000. A correction increases fare and total by 0.01 and leaves all other retained fields unchanged. The inputs and mutations are identical for every system. They are generated corrections rather than observed trip revisions. The public-data builder ignores the inherited payload_bytes=8 configuration field and keeps all eleven source fields, deriving payload size from measured serialized bytes. Canonical records are stored as JSON-encoded strings in the common value column. The measurements therefore concern neither native nested-object mutation nor queries on separate fields.

4.4. Timing, correctness and uncertainty. Preparation and repository construction precede the timed operations, followed by correctness verification and analysis. We give import, commit, diff and checkout separate timing boundaries. Under the common output contract, diff returns sorted keys and hashes of their old/new values. Table 5 sets out which measurements cover algorithm work, interface costs or the whole worker. Correctness checks and decision probes are outside the operation timers.

Table 5
Timing boundaries and interpretation of the measurements

Quantity	Revon / same-representation control	Dolt CLI workflow	Interpretation
Diff latency	Version API, selected path and canonical output	Native JSON-diff command, parsing and canonical output	Complete interface contract; no engine-only ranking
Phase probe	Separate ancestry/count probe; includes detached-view copying	Command, JSON parse and output-construction timers	Neither probe nor command isolates pure tree-engine work
Correctness	All states, reverse/identity, integrity and reopen outside timers	Observed native rows and historical checkout outside timers	Observed values compared with an independently constructed oracle
Resources	Worker plus sampled children, including oracle/setup	Worker plus sampled Dolt processes	10 ms samples; short processes/peaks may be missed
Persistent bytes	Windows: closed files; Linux: before close/compaction	Repository directory files	Includes metadata/indexes; not isolated algorithm overhead

We check the observed endpoint diff and every historical state against workload oracles constructed separately from storage code. Revon also checks reverse and identity requests, object integrity and repository reopening. Reconstructing the plan and semantic hashes lets the audits establish internal consistency. It does not repeat the timing experiment.

For each fixed Linux history, seven trials create fresh repositories, with paired percentile-bootstrap intervals used for timing estimates. A Windows case instead makes seven cached queries within one fresh repository. Its medians across generated histories are reported separately from those across repository repetitions. Those are different experimental units, and neither campaign samples a population of machines.

Sampling covers setup, verification and storage operations for the worker and its observed descendants. CPU and memory measurements thus include work outside the storage implementation. Primary Linux read-counter records do not distinguish supported zero values from missing counters. The Linux supplement and Windows sampler do distinguish them. Windows process I/O bytes nevertheless cannot measure direct physical-disk traffic.

Repository footprint includes persistent metadata and representation costs. Linux directory size is recorded before closing or compacting. Windows size is recorded after closing and the specified verification steps. These conditions differ, so we report the measurements separately; neither isolates the tree's overhead.

4.5. Frozen threshold and geometry controls. The auxiliary Windows calibration selected its largest candidate, T=16,384, without establishing a crossover. The Linux controls use that threshold unchanged. A separate Windows selector experiment, described below, has its own fixed design, executed source and disjoint calibration/validation seeds. Its threshold results are analysed separately.

At 10,000 rows, the Linux control compares forced log, forced Merkle and hybrid with T=16,384 over intervals of 8,192 and 32,768 accumulated operations. This control is not chosen or retuned from Windows outcomes.

The Linux geometry comparison uses one fixed spread workload and five configurations. The Windows sensitivity comparison uses three generated histories and matched equal-capacity geometries. Bucket capacity and operation threshold are distinct parameters. Results describe the settings tested without establishing a geometry that is generally optimal.

4.6. Windows calibration and validation. In the Windows selector comparison, N=12,000, H=8, payload=32 bytes and b=8,d=4 are fixed. We use two calibration seeds and three different validation seeds, crossing spread/repeated-key histories with 64, 512 and 4,096 changes per commit. Each case shares a durable representation across forced log, forced Merkle and hybrid thresholds T=512/4,096/16,384/65,536. Configuration order is shuffled deterministically. After two warm-ups, we measure seven query blocks. Every case starts in a fresh worker, although its queries share resident objects and the filesystem cache. These are not cold-cache measurements.

Calibration normalizes each hybrid case median by the lower forced-path median. We then minimize the equally weighted mean across cases, choosing the smaller T in an exact tie. The threshold and its UTC selection time are recorded before the first validation worker begins; validation cannot select another winner. Sensitivity testing varies rows, history interval, payload, geometry and routing in stages specified in advance. Keeping accumulated operations fixed links interval length to changes per commit, preventing a fully factorial design. The protocols and appendices identify all seeds and matched controls.

4.7. Public histories and repeated scale experiments. The Wine Quality source provides 6,497 red/white records, with keys formed from colour and source row ordinal [11]. For each of three generated histories, we apply eight batches of 64 seeded corrections to the quality field. These are not observed revisions of specimens. The CO2 package supplies published NOAA-derived CSV revisions [12]. Before freezing the experiment, we acquire the eleven most recent snapshots that changed the file and exclude one header-only publication failure. Ten populated snapshots remain, giving nine observed transitions from 812 initial records. We retain unnamed trailing columns as extra fields and use dates to identify records. CSV ordering and formatting do not affect state comparison. This is a repository publication history, rather than NOAA transaction logs or evidence about atmospheric science.

The three CO2 repositories repeat the same observed sequence. Wine and scale repositories use distinct generated seeds instead. We run each public source through Revon-M, T=4,096 Revon-H and native Dolt JSON diff, rotating model order. For scale, H=4, C=64 and payload=32 are fixed at 100,000 and 1,000,000 records, with three seeds per size/system. We report descriptive medians and full ranges across repository medians. The metadata retains source URLs, licences, acquisition versions, checksums and transformations. Restricting acquisition to a bounded publication history made it feasible on the available host without a bulk Software Heritage collection.

The execution-count matrix is in Appendix B, Table 17; the TLC checksums and Linux public-data protocol are in Table 16. The main text supplies timing boundaries and experimental units so that interpreting results does not require reconstructing the entire execution manifest.

5. Linux Results.

5.1. Hybrid versus forced Merkle. Revon-M and Revon-H have the same persistent structures and changesets. In this ablation, only the requested differencing strategy changes. All nine repeated-trial comparisons are reported in Figure 3 and Table 6. The hybrid interval excludes parity in eight cases; for dense S3 it includes one. Log access helps the sparse and repeated-key histories tested, but that benefit concerns diff execution, not a reduction in stored data. It does not establish that switching benefits every workload.

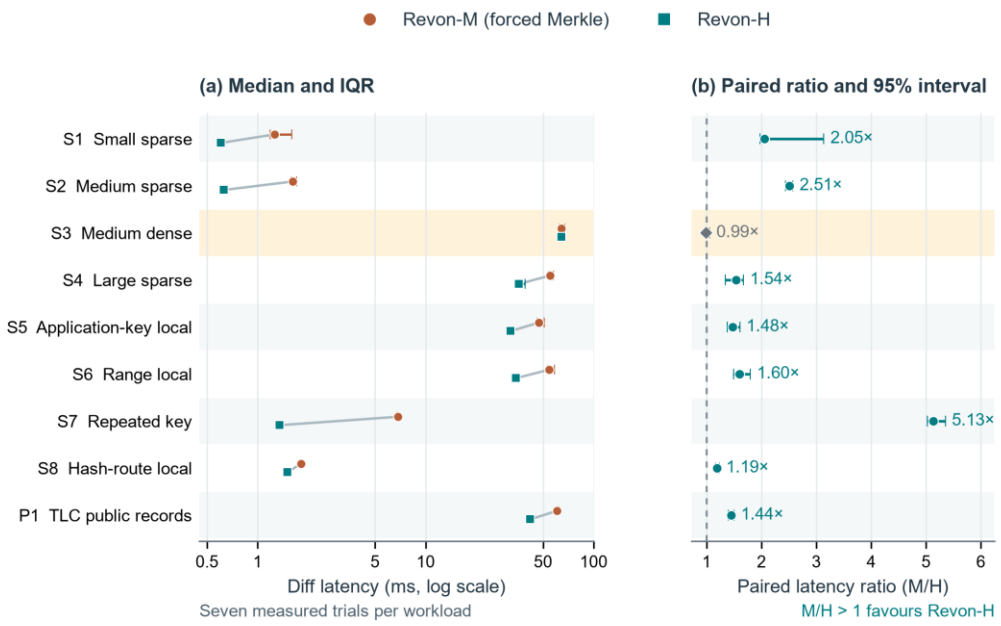


Fig 3. Linux comparison of Revon-M and Revon-H diffs. In (a), connected workload pairs show median latency and interquartile ranges on a logarithmic axis. In (b), median matched-trial M/H ratios have paired percentile-bootstrap 95% intervals from 20,000 resamples. The dashed line is parity. Seven measured trials contribute to each workload. S3 is highlighted because its interval contains parity; the remaining eight favour H. The median of trial ratios need not match the ratio of median latencies. Table 4 gives workload definitions. The intervals describe repeated timings for fixed histories on one VM, not variability between independent datasets or machines.

Table 6
Paired M/H diff ablation, with latency in milliseconds and median trial ratio [95% interval]

ID	M (ms)	H (ms)	M/H ratio [interval]	H path
S1	1.259	0.604	2.05 [1.97, 3.14]	log
S2	1.620	0.629	2.51 [2.44, 2.57]	log
S3	64.135	64.183	0.99 [0.97, 1.06]	merkle
S4	54.897	35.749	1.54 [1.34, 1.67]	log
S5	47.253	31.967	1.48 [1.38, 1.60]	log
S6	54.352	34.537	1.60 [1.50, 1.80]	log
S7	6.862	1.345	5.13 [5.02, 5.35]	log
S8	1.817	1.503	1.19 [1.17, 1.23]	log
P1	60.597	41.964	1.44 [1.39, 1.50]	log

P1 has median diff latency of 41.964 ms under Revon-H and 60.597 ms under forced Merkle, using identical eleven-field public records and generated edits. On S3, H chooses Merkle for the interval's 10,000 accumulated operations. Its near-parity timing is consistent with a shared diff path and the extra selection work in H.

5.2. Threshold control and adverse result. Figure 4 and Table 7 retain both controls, including the slower selection. Hybrid uses log for 8,192 operations and switches to Merkle for 32,768. Forced log is faster in all seven matched measured trials at the larger count. For these cases, operation count alone therefore fails to choose the fastest path. This Linux test of the fixed threshold leaves the best threshold undecided.

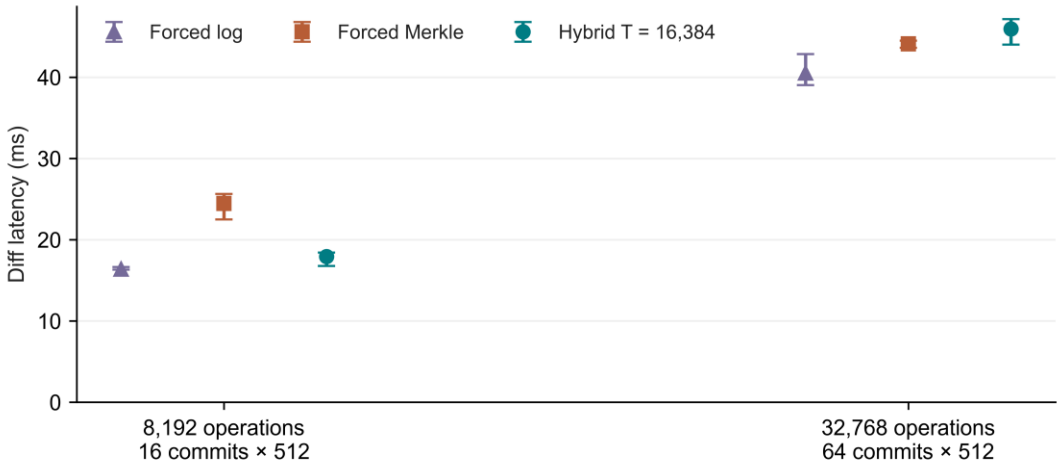


Fig 4. Fixed T=16,384 policy in the Linux controls. Points are medians and bars are IQRs from seven measured trials per path; the labels identify history lengths

Table 7
Diff medians in milliseconds for the fixed-threshold controls

Operations / history	Forced log	Forced Merkle	Hybrid	Selected path
8,192 / H=16	16.364	24.466	17.926	log
32,768 / H=64	40.520	44.120	45.944	merkle

5.3. Complete workflow comparison. Complete canonical-diff workflows are compared in Table 8 through Revon-H's API and Dolt's SQL/CLI interface. Each of the nine paired Dolt/H intervals is above one. Timings include process startup, data structures, language, serialization, query execution and resident state, so storage-engine performance cannot be isolated from the ratios. Figure 5 reports commit and checkout independently, showing how costs differ between operations.

Table 8
Dolt SQL/CLI and Revon-H diff latency in milliseconds, with paired ratio [95% interval]

ID	Dolt (ms)	H (ms)	Dolt/H ratio [interval]
S1	90.704	0.604	148.15 [139.77, 160.15]
S2	95.515	0.629	148.70 [140.08, 150.80]
S3	297.617	64.183	4.70 [4.54, 4.77]
S4	152.472	35.749	4.25 [3.84, 4.34]
S5	125.594	31.967	3.73 [3.71, 3.87]
S6	135.765	34.537	3.88 [3.69, 4.12]
S7	97.451	1.345	71.73 [68.95, 74.32]
S8	110.966	1.503	71.24 [68.83, 77.06]
P1	168.576	41.964	4.02 [3.77, 4.07]

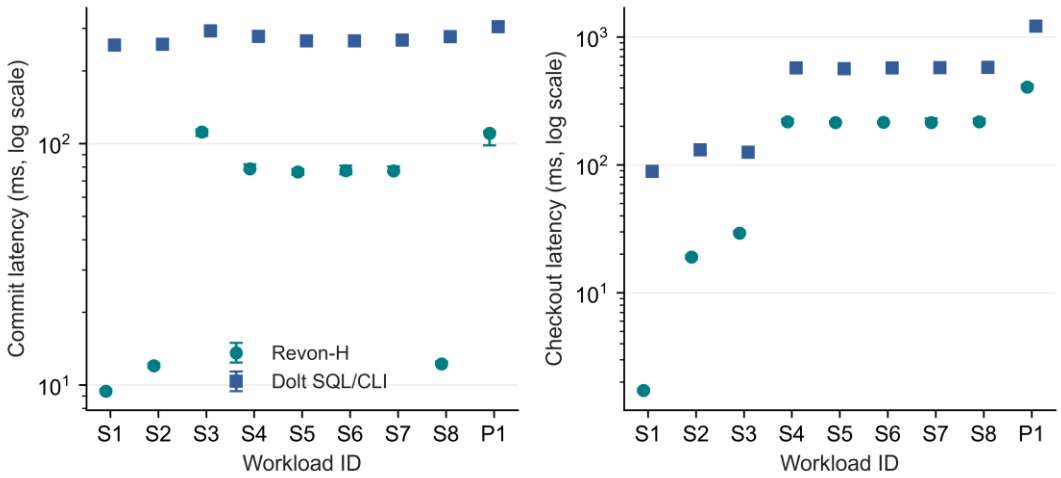


Fig 5. Linux incremental-commit and full-checkout timings for Revon-H and Dolt SQL/CLI. Points represent medians and bars represent IQRs over seven trials, with logarithmic latency axes

5.4. Storage and memory costs. Figure 6 and Table 9 report persistent directory size and whole-worker peak RSS alongside latency. M and H both retain content-addressed changesets and trie data. Selecting log removes no data from that representation; the small directory-size difference comes from the measured workflows. Dolt also uses different serialization, compression and indexing. Lower diff latency can thus come with a substantially larger repository. An efficiency assessment has to include storage and worker resources as well as latency.

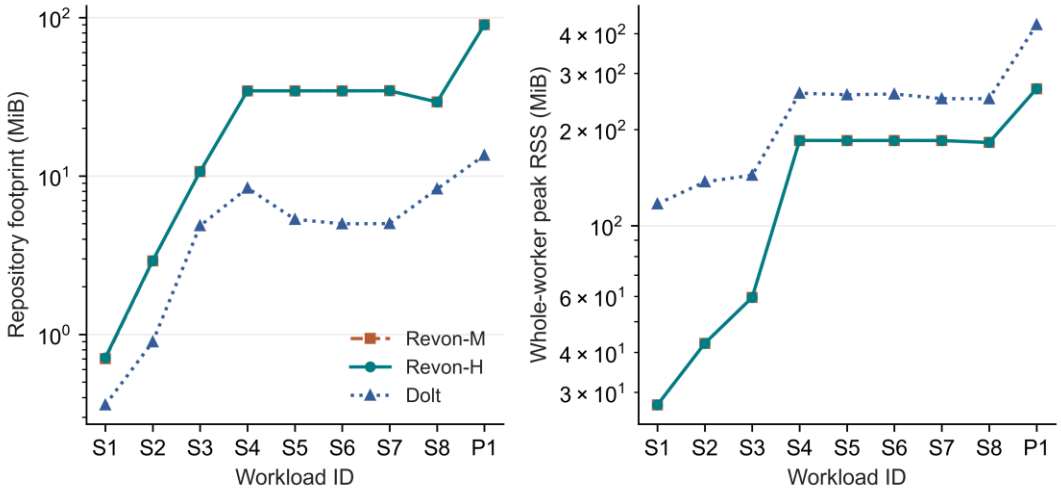


Fig 6. Median pre-compaction repository size and median peak RSS for the whole worker over seven Linux trials, plotted on logarithmic axes. RSS includes shared workload and verification memory

Table 9
Median repository footprint and worker RSS in MiB

ID	M disk	H disk	Dolt disk	H RSS	Dolt RSS
S1	0.7	0.7	0.4	27.4	116.9
S2	2.9	2.9	0.9	42.7	137.1
S3	10.7	10.7	4.8	59.5	143.9
S4	34.4	34.4	8.4	184.9	260.1
S5	34.4	34.4	5.3	184.8	257.2
S6	34.4	34.4	5.0	184.9	258.6
S7	34.5	34.5	5.0	184.8	250.0
S8	29.3	29.3	8.3	182.2	249.9
P1	90.1	90.0	13.5	268.6	426.5

5.5. Million-record feasibility. The Linux million-row diagnostic creates and verifies one fresh repository for each of M, H and Dolt. Verification covers all eleven historical states and the shared diff oracle. The individual observations are listed in Table 10 without IQRs, confidence intervals or rankings. Hybrid selects Merkle because 10,000 accumulated operations exceed T=4,096. These runs demonstrate completion of the bounded workload on the VM, but one observation per system cannot determine relative scaling or an optimal configuration. Section 6.4 presents the separate repeated Windows scale experiment.

Table 10
Single million-record observations, n=1 per system, without comparative inference

System	Import (s)	Commit (ms)	Diff (ms)	Checkout (s)	RSS (GiB)
Dolt	24.902	547.311	817.034	11.454	2.555
Revon-H	32.289	2961.901	1233.601	4.729	2.561
Revon-M	32.454	2979.717	1265.367	4.685	2.561

5.6. Fixed-trie geometry and telemetry. The Linux geometry comparison appears in Figure 7 and Table 11. Configurations b4-d6, b8-d4 and b16-d3 each have 4,096 leaf buckets, giving equal expected occupancy for examining routing shape at that capacity. At fixed b=8, changing depth also changes capacity and the trade-off between traversal and leaf scanning. Only one fixed workload on one VM is used, so the experiment cannot identify a generally optimal geometry.

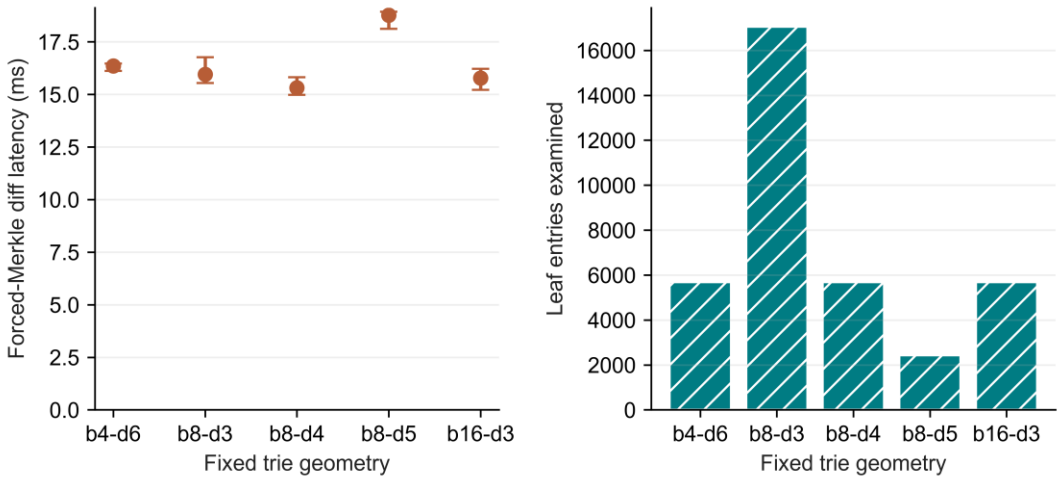


Fig 7. Fixed-trie geometry measured on Linux. Left: forced-Merkle diff medians and IQRs over seven trials. Right: median examined leaf-entry count

Table 11
Work and footprint across geometries for one fixed spread workload

Geometry	Buckets	Diff (ms)	Leaf entries	Node pairs	Disk (MiB)
b4-d6	4096	16.355	5682	3628	4.32
b8-d3	512	15.949	17048	585	6.88
b8-d4	4096	15.316	5682	3729	4.25
b8-d5	32768	18.765	2426	6089	6.18
b16-d3	4096	15.775	5682	3929	8.12

Two isolated-worker telemetry tests pass by distinguishing valid all-zero I/O readings from unavailable counters. The Linux telemetry supplement comprises 27 executions on a fixed 10,000-row workload, with M, H and Dolt each receiving two warm-ups and seven measured trials. Its records keep measured zero read bytes as values rather than blanks. The check establishes correct counter handling, without demonstrating cold-storage behaviour or ruling out missed samples. The primary latency summaries remain a separate analysis.

5.7. Representation baselines. Table 12 uses representative spread workload S4 to give descriptive medians for all six workflows. The GitHub reproducibility archive described in Appendix A provides the full eight-scenario matrix, dispersion between trials, normalized storage and separate SQL/CSV import workflows. Snapshot and Log-only are representation controls in the common harness, not equivalents for all production features. Their full-state storage and operation-replay costs put Revon's measurements in context.

Table 12
Representative S4 workflow medians; the GitHub reproducibility archive supplies the complete eight-scenario matrix

Workflow	Import (s)	Commit (ms)	Diff (ms)	Checkout (ms)	Disk (MiB)
Snapshot	0.071	69.577	624.860	168.158	53.5
Log-only	0.072	2.474	16.483	159.538	5.0
Revon-M	1.488	78.842	54.897	215.661	34.4
Revon-H	1.644	78.590	35.749	217.482	34.4
Dolt SQL	1.663	277.593	152.472	572.585	8.4
Dolt CSV	1.335	273.425	149.325	559.342	7.9

6. Windows Evaluation Results.

6.1. Held-out selector behaviour. Calibration selects the largest candidate, T=65,536, at a normalized score of 1.014. Across 18 held-out histories its mean normalized latency is 1.012, compared with 1.101 for T=4,096. The respective agreements with the lower forced-path median are 18/18 and 12/18. These counts refer to noisy measured medians, not classification against a known optimum. Table 13 and Figure 8 retain every profile group.

Primary Windows intervals contain no more than 32,768 operations. As a result, T=65,536 uses log everywhere in this matrix. Its calibration win does not demonstrate useful switching or establish a global crossover. Timing noise can also produce ratios below one when hybrid and forced execution choose equivalent paths. This result, together with the adverse Linux control, bounds the conclusions about the selector. The threshold selected for the experiment leaves the implementation's default unchanged.

Table 13
Held-out median of three history medians at fixed N/H/payload; no population interval

Locality	Operations	Forced log ms	Forced Merkle ms	H4096 ms	Calibrated ms
spread	512	1.913	4.622	1.907	1.837
spread	4,096	25.714	38.878	26.550	27.312
spread	32,768	92.792	104.71	106.56	91.775
repeated-key	512	0.564	1.661	0.549	0.573
repeated-key	4,096	5.608	14.342	6.311	4.664
repeated-key	32,768	62.290	74.366	77.092	67.721

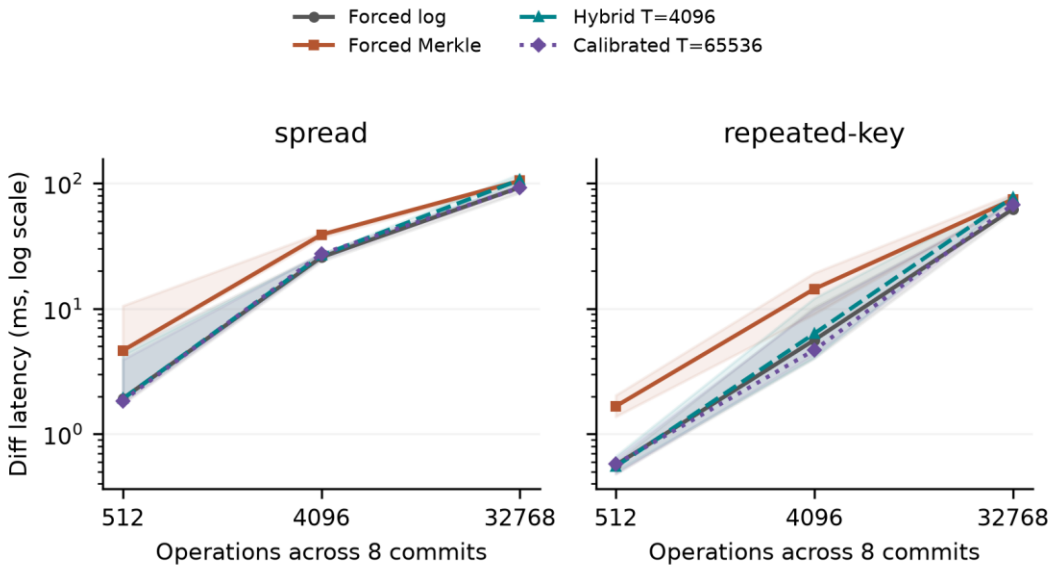


Fig 8. Selector results on held-out histories at N=12,000 and H=8. Lines link the medians of three generated-history medians; shading extends from the minimum to the maximum. Each history provides seven measured cached queries. The shaded range is descriptive, not a 95% confidence interval. The calibrated policy selects log at all plotted operation counts. All forced paths share the durable representation.

6.2. Sensitivity and resource interpretation. The decision probe runs outside diff timing and reads a detached public changeset view. That copying cost means the probe cannot measure selector overhead alone. We use a separate diagnostic to extract the production function prefix for validation, ancestry and internal operation counting, ending before dispatch. Over 18 held-out histories and two thresholds, 252,000 measured warm calls give a median of case/threshold medians of 2.674 microseconds, with a range of 1.611-3.678. The decisions agree with production dispatch. The diagnostic excludes diff traversal and detached-view copying, and measures neither cold-cache behaviour nor a causal correction that can be subtracted from workflow timings. It is distinct from the 93-case workflow campaign.

Across three seeds, the 21 sensitivity cases vary a prespecified factor or a matched control. They include payload=256 bytes, N=24,000, fixed-operation H=16, equal-capacity b4-d6/b16-d3 geometries and a C=8 hash-route/spread pair. The geometry cases change routing geometry while reproducing the base history's state hashes. With operations held fixed, history length trades off against changes per commit, preventing estimation of their independent effects in that comparison. The results describe these profiles and do not choose a generally best geometry.

Median forced-Merkle latency is 47.857 ms for more rows, 40.122 ms for longer histories and 39.950 ms for larger payloads. The b4-d6 and b16-d3 geometries give 43.004 ms and 38.500 ms, respectively. Hash-route locality gives 0.174 ms compared with 0.903 ms for the matched spread control. Each value summarizes three history medians by their median. Per-history timing, selected paths and resource scopes are retained in the summaries. We treat these comparisons as exploratory and make no multiple-testing significance claim.

For T=4,096, median-latency ratios in the three sampler off/on pairs range from 0.950 to 1.206. Sampling-off runs always come before sampling-on runs, with cache state and background activity uncontrolled. This ordering prevents a causal estimate of the sampling penalty, and we subtract no correction from other timings. Resource counters measure

entire workers. Although tests preserve observed zero I/O as a value, Windows counters cannot identify database logical reads or direct physical-disk traffic.

6.3. Public data and interface phases.

Table 14
Wine and CO2 workflows: medians and full ranges of three repository medians; one observed CO2 history is repeated

Source / system	Runs	Diff ms	Range ms	Import s	Persistent MiB
co2/revon-m	3	0.612	0.562-0.661	0.033	1.191
co2/revon-h	3	0.309	0.302-0.318	0.035	1.184
co2/dolt	3	256.98	251.87-262.89	0.634	0.178
wine/revon-m	3	4.281	4.260-4.964	0.189	10.832
wine/revon-h	3	2.247	1.956-2.366	0.191	10.812
wine/dolt	3	296.76	246.75-302.81	0.819	1.360

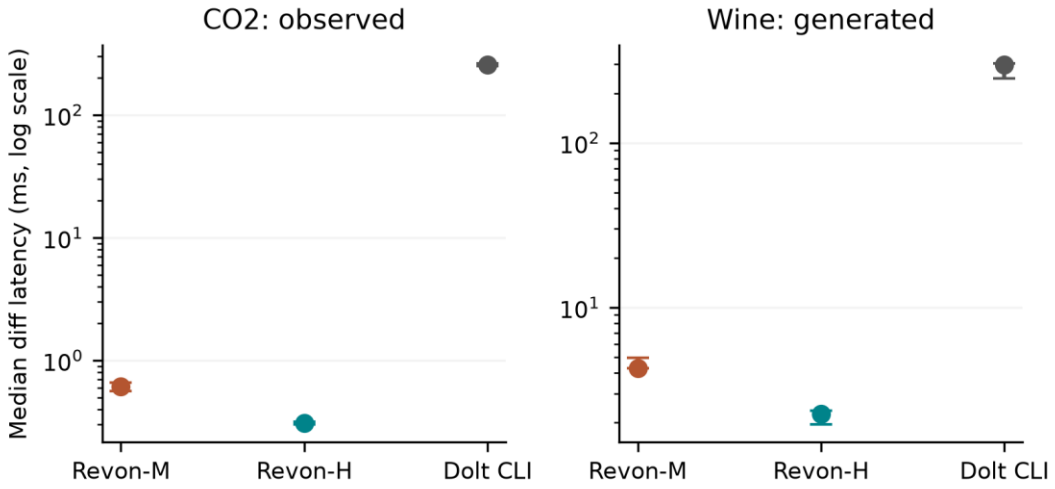


Fig 9. Workflows on public sources. Points show the median of three repository medians and bars show their full range. CO2 repositories repeat one observed publication history; wine repositories use three generated correction histories. Each repository contributes seven cached queries. Different API/CLI boundaries mean these are workflow latencies, not measurements of isolated storage engines.

In co2-observed, median Dolt command time is 256.063 ms, with 0.501 ms for JSON interpretation and 0.503 ms for common output construction. The command takes 99.6% of median complete diff latency. It includes startup, native work and native JSON emission, so the breakdown cannot assign that entire interval to the storage engine or justify subtracting a separately measured startup constant.

In wine-generated, median times are 288.464 ms for the Dolt command, 4.445 ms for JSON interpretation and 3.258 ms for common output construction. The command represents 97.2% of median complete diff latency and includes startup, native work and native JSON emission. As for CO2, these measurements neither isolate the engine's cost nor justify subtracting an independent startup estimate.

6.4. Repeated Windows scale observations.

Table 15
Windows scale repetitions: medians of three seed/repository summaries; full ranges are recorded in the evidence

Rows / system	Import s	Commit ms	Diff ms	Checkout s	Worker RSS GiB
100,000/revon-m	1.116	54.309	5.649	0.209	0.248
100,000/revon-h	1.109	53.044	1.004	0.173	0.242
100,000/dolt	1.662	759.99	283.65	0.662	0.223
1,000,000/revon-m	10.614	88.233	33.694	1.862	1.939
1,000,000/revon-h	10.738	85.362	1.013	1.575	1.925
1,000,000/dolt	11.281	890.97	320.30	3.706	0.976

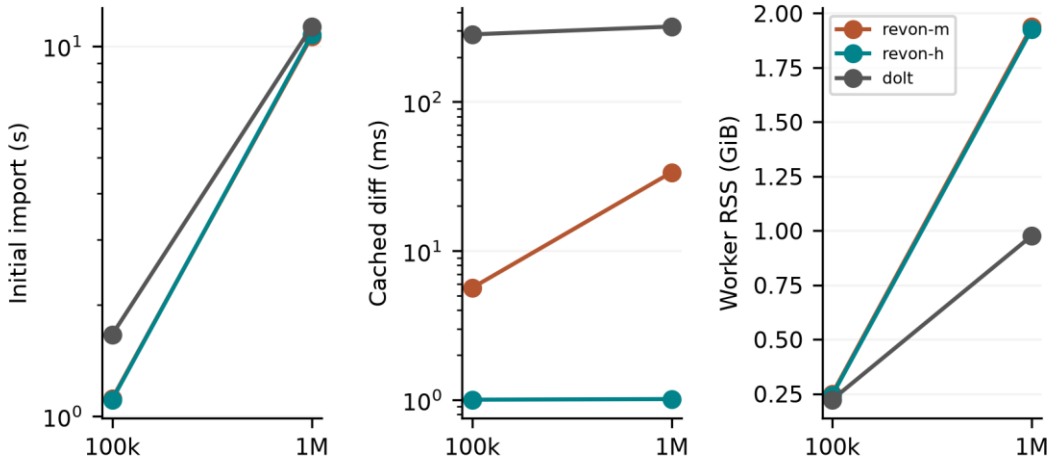


Fig 10. Repeated Windows scale observations with three generated histories and fresh repositories per size/system. Connecting lines show descriptive medians at two sizes, not fitted asymptotic laws. Diff queries use cached state; import and checkout are timed independently. Peak RSS covers workload/oracle generation, storage work, verification and sampled descendants. Linux million-row n=1 diagnostics are from a separate campaign.

We keep the repeated Windows scale analysis separate from the Linux diagnostics. Its three histories use different updates on the same initial synthetic distribution, rather than independent public datasets. The manifest records available memory and background processes, but those conditions are uncontrolled. The observations describe the workflows tested under those conditions; they do not establish a ranking across a population or a general scaling law.

6.5. Answers to the research questions. RQ1 is addressed through the latency, persistent-storage and worker-resource trade-offs of complete workflows. API/CLI boundaries prevent reading these as an engine-only ranking. For RQ2, log improves several fixed-history timings, but Windows calibration chooses a policy that selects it throughout the tested intervals. The benefit of adaptive switching is therefore unestablished. For RQ3, controlled histories expose payload, routing and geometry effects within the staged design, without identifying a universal layout. Observed CO2 transitions differ from generated wine corrections and synthetic updates, and each source keeps its own scope limits.

7. Discussion. The workflow measurements answer RQ1 at an application's interface. Linux and Windows experiments both measure import, commit, diff, checkout and persistent footprint, with campaign rows analysed separately. Snapshot and Log-only expose representation costs, while forced M, H and log comparisons leave Revon's representation fixed. Dolt provides a richer SQL/versioning interface. Its separate command phases allow closer inspection of that workflow without equating native tree work to an isolated Python call.

RQ2 calls for testing whether a parameter selected on one set of histories transfers to others. We hold primary N, H, payload and geometry fixed in the Windows split and record selection before validation. The largest candidate is equivalent to always-log across the measured range. Dense-case parity and the adverse Linux threshold choice show why choosing the fastest path cannot be taken for granted. A more complex selector would need separate calibration followed by untouched testing. These findings do not justify changing the default threshold.

For RQ3, equal capacity can still allow different traversal depths and ancestor-reconstruction costs, even with equal expected occupancy under uniform routing. Encoding payloads, examining touched leaves and constructing output also contribute to latency. Windows cases use separate generated seeds with matched controls; Linux geometry uses one fixed history. Neither design estimates all interactions or an optimum for a population.

Consistent value semantics are necessary for comparing diffs. Revon's detached inputs and outputs prevent mutable aliases from changing retained history. Under v1 encoded identity, true differs from 1 and 1 differs from 1.0, with object encoding and the on-disk schema preserved. The Linux trials exercise these semantics on the supplied histories. Unit tests also check null versus absence, nested ownership, unsupported values and rollback.

8. Limitations And Future Work. External validity: the Linux campaign ran on one cloud VM and Windows on one local host, both operated by the authors. These are not independent-party replications or controlled OS comparisons. Hardware, runtime, virtualization and storage vary together. Filesystem caches and background services remain active, and the Windows manifest records limited available RAM and process conditions. The study does not represent an entirely idle host or cold-cache operation.

Workload validity: the TLC histories contain generated updates to one monthly release. Linux scale diagnostics have n=1 per system. Windows validation uses three held-out generated seeds per primary profile, generated wine corrections and one observed CO2 publication sequence. Its three CO2 repositories replay the same sequence rather than sampling independent histories. A header-only revision is removed before timing; unnamed columns are retained without scientific interpretation. Synthetic scale histories start from a common distribution. Wider generalization requires further domains, observed histories and hardware.

Implementation scope: the fixed-geometry trie eagerly loads retained nodes, changesets and commits, with no eviction. Revon is limited to one writer and linear history. The evaluation does not cover named branches, merge commits, conflict handling, schema evolution, remotes, ordered scans, crash fault injection or concurrent workloads. Atomic SQLite transactions alone do not demonstrate all the durability and operational guarantees needed for production.

Measurement and inference: Dolt command time contains process startup, native work and JSON emission, while worker RSS includes oracle and history state. Ten-millisecond sampling can miss brief peaks or processes. Windows I/O counters measure neither physical-disk traffic nor database logical reads, and sampler off/on comparisons confound

order with cache conditions. Linux bootstrap intervals characterize repeated fixed-history timings. Windows three-history ranges are descriptive, without multiple-comparison correction. Checks of sources and semantic hashes verify internal consistency, not independent timing reproduction.

Selection and design: the primary Windows calibration considers four thresholds, two generated seeds and the specified profiles. Its largest candidate, T=65,536, never switches over the tested range, leaving broader adaptive-selection usefulness unresolved. The sensitivity design is staged rather than fully factorial: at fixed operation count H trades off against density, and hash-route cases must use their matched low-density control. A different selector or crossover needs a separately fixed experiment.

Future storage changes could include binary nodes, value chunks, packed objects, compression and reachability-based garbage collection. Ordered Merkle indexing or content-defined chunking could support a different approach to range operations. Semantic and performance tests would be needed for each change; the fixed-trie results do not predict their effects.

9. Conclusion. Revon offers two semantically consistent diff paths for a content-addressed fixed-trie history persisted atomically in SQLite. The contribution is an inspectable prototype with empirical measurements of workflow and selection costs. Linux evidence covers 516 executions and a separate 72-execution supplement, including the uncertain dense-case result and adverse threshold choice. The Windows experiments contribute 93 worker cases and 3,402 checked queries across held-out histories, Wine Quality and CO2 sources, interface phases and repeated scale runs. The calibrated T=65,536 policy uses log throughout the tested primary matrix, leaving the optimal crossover unresolved. The recorded workloads demonstrate conditional trade-offs, bounded by API/CLI, sampling, dataset and hardware limits. They establish neither a universal speed advantage nor independent-party reproduction.

Funding: The authors received no funding for this study.

Competing interests: There are no competing interests to declare.

Ethics and consent to participate: This computational study recruited no participants and involved no human or animal intervention. It uses synthetic histories and public de-identified TLC records. The authors claim no institutional ethics approval or exemption.

Consent for publication: The paper contains no identifiable participant material. Consent for publication is therefore not applicable.

Data and code availability: Public inputs comprise NYC TLC taxi records [24], UCI Wine Quality [11] and the published CO2 package [12]. Code, executed source snapshots and experimental evidence are provided at https://github.com/atharvasheersh/Revon. The paper/ directory holds the Linux reproducibility archive and the Windows reproducibility archive. We do not redistribute TLC input records or generated databases. Source metadata records the licences and transformations; Appendix A explains how to access the materials.

Acknowledgment. We thank the open-source Dolt project and the researchers in dataset versioning whose work informed our experimental design.

REFERENCES

[1] M. Akhlaghi et al. (2021). Toward long-term and archivable reproducibility, Computing in Science & Engineering, vol. 23, no. 3, pp. 82-91, https://doi.org/10.1109/MCSE.2021.3072860. [Online]. Available: https://arxiv.org/abs/2006.03018.

[2] M. Armbrust et al. (2020). Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores, Proc. VLDB Endowment, vol. 13, no. 12, pp. 3411-3424, https://doi.org/10.14778/3415478.3415560.

[3] Attic Labs. Noms Technical Overview. GitHub. [Online]. Available: https://github.com/attic-labs/noms/blob/master/doc/intro.md.

[4] A. Auvolat and F. Taïani (2019). Merkle Search Trees: Efficient State-Based CRDTs in Open Networks, in Proc. 38th IEEE Int. Symp. Reliable Distributed Syst. (SRDS), pp. 221-230, https://doi.org/10.1109/SRDS47363.2019.00032.

[5] J. Benet (2014). IPFS: Content Addressed, Versioned, P2P File System, arXiv:1407.3561, [Online]. Available: https://arxiv.org/abs/1407.3561.

[6] A. Bhardwaj et al. (2015). DataHub: Collaborative Data Science and Dataset Version Management at Scale, in Proc. 7th Biennial Conf. Innovative Data Syst. Research (CIDR), Open copy: https://www.cidrdb.org/cidr2015/Papers/CIDR15_Paper18.pdf.

[7] S. Bhattacherjee, A. Chavan, S. Huang, A. Deshpande, and A. Parameswaran (2015). Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff, Proc. VLDB Endowment, vol. 8, no. 12, pp. 1346-1357, https://doi.org/10.14778/2824032.2824035.

[8] G. S. Brodal, S. Sioutas, K. Tsakalidis, and K. Tsichlas (2020). Fully persistent B-trees, Theoretical Computer Science, vol. 841, pp. 10-26, https://doi.org/10.1016/j.tcs.2020.06.027. [Online]. Available: https://cs.au.dk/~gerth/papers/tcs20.pdf.

[9] P. Buneman, S. Khanna, and W.-C. Tan (2001). Why and where: A characterization of data provenance, in Database Theory: ICDT Lecture Notes in Computer Science, vol. 1973, 2001, pp. 316-330, https://doi.org/10.1007/3-540-44503-X_20. [Online]. Available: https://www.pure.ed.ac.uk/ws/files/16509989/Why_and_Where_A_Characterization_of_Data_Provenance.pdf.

[10] S. Chacon and B. Straub (2014). Git Internals: Git Objects, in Pro Git, 2nd ed., [Online]. Available: https://git-scm.com/book/en/v2/Git-Internals-Git-Objects.

[11] P. Cortez, A. Cerdeira, F. Almeida, T. Matos and J. Reis (2009). Wine Quality [Dataset], UCI Machine Learning Repository, https://doi.org/10.24432/C56S3T. CC BY 4.0.

[12] DataHub Contributors (2026). CO2 PPM - Trends in Atmospheric Carbon Dioxide, NOAA-derived published data package, accessed October https://github.com/datasets/co2-ppm. ODC-PDDL-1.0; acquired revision identifiers and file hashes are retained in the experiment manifest.

[13] DoltHub (2026). Dolt Storage Engine: Block Store, Dolt Documentation, accessed Oct. 2, [Online]. Available: https://www.dolthub.com/docs/architecture/storage-engine/block-store/.

[14] J. R. Driscoll, N. Sarnak, D. D. Sleator, and R. E. Tarjan (1989). Making data structures persistent, J. Comput. Syst. Sci., vol. 38, no. 1, pp. 86-124, https://doi.org/10.1016/0022-0000(89)90034-2.

[15] J. Edwards and T. Petricek (2025). Baseline: Operation-based evolution and versioning of data, arXiv:2512.09762, preprint, updated 2026. [Online]. Available: https://arxiv.org/abs/2512.09762.

[16] A. González-Cebrián, M. Bradford, A. E. Chis, and H. González-Vélez (2024). Standardised versioning of datasets: A FAIR-compliant proposal, Scientific Data, vol. 11, art. 358, https://doi.org/10.1038/s41597-024-03153-y. [Online]. Available: https://www.nature.com/articles/s41597-024-03153-y.

[17] H. Gou, Z. Zhang, Y. Sun, P. Xu, F. Tian, L. Wang, and J. Wang (2026). Git4Data: Database-native version control for AI agents, arXiv:2609.02106, preprint. [Online]. Available: https://arxiv.org/abs/2609.02106.

[18] A. Guo, J. Li, P. Sukprasert, S. Khuller, A. Deshpande, and K. Mukherjee (2024). To store or not to store: A graph theoretical approach for dataset versioning, in Proc. IEEE Int. Parallel and Distributed Processing Symposium (IPDPS), pp. 479-493, https://doi.org/10.1109/IPDPS57955.2024.00049. [Online]. Available: https://arxiv.org/abs/2402.11741.

[19] S. Huang, L. Xu, J. Liu, A. J. Elmore, and A. Parameswaran (2017). OrpheusDB: Bolt-on Versioning for Relational Databases, Proc. VLDB Endowment, vol. 10, no. 10, pp. 1130-1141, https://doi.org/10.14778/3115404.3115417.

[20] M. Maddox et al. (2016). Decibel: The Relational Dataset Branching System, Proc. VLDB Endowment, vol. 9, no. 9, pp. 624-635, https://doi.org/10.14778/2947618.2947619.

[21] F. McSherry, D. G. Murray, R. Isaacs, and M. Isard (2013). Differential dataflow, in Proc. 6th Biennial Conf. Innovative Data Systems Research (CIDR), [Online]. Available: https://www.cidrdb.org/cidr2013/Papers/CIDR13_Paper111.pdf.

[22] R. C. Merkle (1988). A digital signature based on a conventional encryption function, in Advances in Cryptology: CRYPTO '87, Lecture Notes in Computer Science, vol. 293, pp. 369-378, https://doi.org/10.1007/3-540-48184-2_32.

[23] A. Muthitacharoen, B. Chen, and D. Mazières (2001). A low-bandwidth network file system, in Proc. 18th ACM Symp. Operating Syst. Principles (SOSP), pp. 174-187, https://doi.org/10.1145/502034.502052.

[24] New York City Taxi and Limousine Commission (2024). TLC Trip Record Data: Yellow Taxi, January public dataset, accessed Oct. 1, 2026. [Online]. Available: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page.

[25] N. Saquib, C. Krintz, and R. Wolski (2023). Replicated versioned data structures for wide-area distributed systems, IEEE Transactions on Parallel and Distributed Systems, vol. 34, no. 1, pp. 207-224, https://doi.org/10.1109/TPDS.2022.3217969. [Online]. Available: https://racelab.cs.ucsb.edu/papers/tpds22-versioned.pdf.

[26] M. E. Schüle, L. Karnowski, J. Schmeißer, B. Kleiner, A. Kemper, and T. Neumann (2019). Versioning in main-memory database systems: From MusaeusDB to TardisDB, in Proc. 31st Int. Conf. Scientific and Statistical Database Management (SSDBM), pp. 169-180, https://doi.org/10.1145/3335783.3335792. [Online]. Available: https://www-db.cs.tum.edu/~schuele/data/tardis.pdf.

[27] A. Seering, P. Cudré-Mauroux, S. Madden, and M. Stonebraker (2012). Efficient versioning for scientific array databases, in Proc. IEEE 28th Int. Conf. Data Engineering (ICDE), pp. 1013-1024, https://doi.org/10.1109/ICDE.2012.102. [Online]. Available: https://dspace.mit.edu/entities/publication/a5c2c050-6abd-47c1-8bee-4f8c51b6a15f.

[28] T. Sehn (2024). Dolt's Storage Engine, DoltHub Blog, Feb. 29, [Online]. Available: https://www.dolthub.com/blog/2024-02-29-storage-engine/.

[29] R. Shraga and R. J. Miller (2023). Explaining Dataset Changes for Semantic Data Versioning with Explain-Da-V, Proc. VLDB Endow., vol. 16, no. 6, pp. 1587-1600, https://doi.org/10.14778/3583140.3583169.

[30] E. Soroush and M. Balazinska (2013). Time travel in a scientific array database, in Proc. IEEE 29th Int. Conf. Data Engineering (ICDE), pp. 98-109, https://doi.org/10.1109/ICDE.2013.6544817. [Online]. Available: https://scidb.cs.washington.edu/paper/ICDE13_conf_full_422.pdf.

[31] A. Twigg, A. Byde, G. Miłos, T. Moreton, J. Wilkes, and T. Wilkie (2011). Stratified B-trees and versioning dictionaries, in Proc. 3rd USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage), [Online]. Available: https://arxiv.org/abs/1103.4282.

[32] S. Wang et al. (2018). ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications, Proc. VLDB Endowment, vol. 11, no. 10, pp. 1137-1150, https://doi.org/10.14778/3231751.3231762.

[33] J. Zhao and S. Krishnan (2024). Compression and in-situ query processing for fine-grained array lineage, arXiv:2405.17701, [Online]. Available: https://arxiv.org/abs/2405.17701.

[34] J. Klump, H. Pampel, L. Rothfritz, and D. Strecker (2024). Recommendations on Data Versioning. Zenodo. https://doi.org/10.5281/zenodo.13743875

[35] D. Trautwein, A. Raman, G. Tyson, I. Castro, W. Scott, M. Schubotz, et al. (2022). Design and evaluation of IPFS. Proceedings of the ACM SIGCOMM 2022 Conference, pp. 739-752. https://doi.org/10.1145/3544216.3544232

[36] A. Arasu, B. Chandramouli, J. Gehrke, E. Ghosh, D. Kossmann, J. Protzenko, et al. (2021). FastVer: Making Data Integrity a Commodity. Proceedings of the 2021 International Conference on Management of Data, pp. 89-101. https://doi.org/10.1145/3448016.3457312

[37] E.D. Demaine, S. Langerman, E. Price (2010). Confluently Persistent Tries for Efficient Version Control. Algorithmica, vol. 57, no. 3, pp. 462-483. https://doi.org/10.1007/S00453-008-9274-Z

[38] S. Lanka, E. Mays (1991). Fully persistent B+-trees. ACM SIGMOD Record, vol. 20, no. 2, pp. 426-435. https://doi.org/10.1145/119995.115861

[39] G.S. Brodal, C.M. Rysgaard, R. Svenning (2023). External Memory Fully Persistent Search Trees. Proceedings of the 55th Annual ACM Symposium on Theory of Computing, pp. 1410-1423. https://doi.org/10.1145/3564246.3585140

[40] C. Allain, B. Clément, A. Moine, G. Scherer (2024). Snapshottable Stores. Proceedings of the ACM on Programming Languages, vol. 8, no. ICFP, pp. 338-369. https://doi.org/10.1145/3674637

[41] A. Miller, M. Hicks, J. Katz, E. Shi (2014). Authenticated data structures, generically. Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, pp. 411-423. https://doi.org/10.1145/2535838.2535851

[42] V. Salis, D. Spinellis (2019). RepoFS: File system view of Git repositories. SoftwareX, vol. 9, pp. 288-292. https://doi.org/10.1016/j.softx.2019.03.007

[43] A. Bhardwaj, A. Deshpande, A.J. Elmore, D. Karger, S. Madden, A. Parameswaran, et al. (2015). Collaborative data analytics with DataHub. Proceedings of the VLDB Endowment, vol. 8, no. 12, pp. 1916-1919. https://doi.org/10.14778/2824032.2824100

[44] A. Chavan, A. Deshpande (2017). DEX. Proceedings of the 2017 ACM International Conference on Management of Data, pp. 171-186. https://doi.org/10.1145/3035918.3064056

[45] J.F. Roddick (1995). A survey of schema versioning issues for database systems. Information and Software Technology, vol. 37, no. 7, pp. 383-393. https://doi.org/10.1016/0950-5849(95)91494-K

[46] G. Ozsoyoglu, R. Snodgrass (1995). Temporal and real-time databases: a survey. IEEE Transactions on Knowledge and Data Engineering, vol. 7, no. 4, pp. 513-532. https://doi.org/10.1109/69.404027

[47] C. De Castro, F. Grandi, M.R. Scalas (1997). SCHEMA VERSIONING FOR MULTITEMPORAL RELATIONAL DATABASES. Information Systems, vol. 22, no. 5, pp. 249-290. https://doi.org/10.1016/S0306-4379(97)00017-3

[48] T.J. Green, G. Karvounarakis, V. Tannen (2007). Provenance semirings. Proceedings of the twenty-sixth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems, pp. 31-40. https://doi.org/10.1145/1265530.1265535

[49] Y.L. Simmhan, B. Plale, D. Gannon (2005). A survey of data provenance in e-science. ACM SIGMOD Record, vol. 34, no. 3, pp. 31-36. https://doi.org/10.1145/1084805.1084812

[50] L. Carata, S. Akoush, N. Balakrishnan, T. Bytheway, R. Sohan, M. Seltzer, et al. (2014). A primer on provenance. Communications of the ACM, vol. 57, no. 5, pp. 52-60. https://doi.org/10.1145/2596628

[51] B. Glavic (2021). Data Provenance. Foundations and Trends in Databases, vol. 9, no. 3-4, pp. 209-441. https://doi.org/10.1561/1900000068

[52] Y. Amsterdamer, D. Deutch, V. Tannen (2011). Provenance for aggregate queries. Proceedings of the thirtieth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems, pp. 153-164. https://doi.org/10.1145/1989284.1989302

[53] F. Jamil, A. Khan, A. Anjum, M. Ahmed, F. Jabeen, N. Javaid (2018). Secure provenance using an authenticated data structure approach. Computers & Security, vol. 73, pp. 34-56. https://doi.org/10.1016/j.cose.2017.10.005

[54] M.D. Wilkinson, M. Dumontier, I.J. Aalbersberg, G. Appleton, M. Axton, A. Baak, et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data, vol. 3, no. 1, article 160018. https://doi.org/10.1038/sdata.2016.18

[55] B. Pan, N. Stakhanova, S. Ray (2023). Data Provenance in Security and Privacy. ACM Computing Surveys, vol. 55, no. 14s, pp. 1-35. https://doi.org/10.1145/3593294

[56] L. Moreau, P. Groth, J. Cheney, T. Lebo, S. Miles (2015). The rationale of PROV. Journal of Web Semantics, vol. 35, pp. 235-257. https://doi.org/10.1016/j.websem.2015.04.001

[57] K. Wolstencroft, S. Owen, O. Krebs, Q. Nguyen, N.J. Stanford, M. Golebiewski, et al. (2015). SEEK: a systems biology data and model management platform. BMC Systems Biology, vol. 9, no. 1, article 33. https://doi.org/10.1186/s12918-015-0174-y

[58] D. Comer (1979). Ubiquitous B-Tree. ACM Computing Surveys, vol. 11, no. 2, pp. 121-137. https://doi.org/10.1145/356770.356776

[59] Y.S. Kwong, D. Wood (1980). On B-trees. Proceedings of the 1980 ACM SIGMOD international conference on Management of data - SIGMOD '80, pp. 207. https://doi.org/10.1145/582250.582282

[60] W.J. Hansen (1981). A Cost Model for the Internal Organization of B+-Tree Nodes. ACM Transactions on Programming Languages and Systems, vol. 3, no. 4, pp. 508-532. https://doi.org/10.1145/357146.357152

[61] M.A. Bender, E.D. Demaine, M. Farach-Colton (2005). Cache-Oblivious B-Trees. SIAM Journal on Computing, vol. 35, no. 2, pp. 341-358. https://doi.org/10.1137/S0097539701389956

[62] P. O'Neil, E. Cheng, D. Gawlick, E. O'Neil (1996). The log-structured merge-tree (LSM-tree). Acta Informatica, vol. 33, no. 4, pp. 351-385. https://doi.org/10.1007/s002360050048

[63] F. Chang, J. Dean, S. Ghemawat, W.C. Hsieh, D.A. Wallach, M. Burrows, et al. (2008). Bigtable. ACM Transactions on Computer Systems, vol. 26, no. 2, pp. 1-26. https://doi.org/10.1145/1365815.1365816

[64] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, et al. (2007). Dynamo. Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles, pp. 205-220. https://doi.org/10.1145/1294261.1294281

[65] M. Rosenblum, J.K. Ousterhout (1992). The design and implementation of a log-structured file system. ACM Transactions on Computer Systems, vol. 10, no. 1, pp. 26-52. https://doi.org/10.1145/146941.146943

[66] L. Lu, T.S. Pillai, H. Gopalakrishnan, A.C. Arpaci-Dusseau, R.H. Arpaci-Dusseau (2017). WiscKey. ACM Transactions on Storage, vol. 13, no. 1, pp. 1-28. https://doi.org/10.1145/3033273

[67] H.T. Vo, S. Wang, D. Agrawal, G. Chen, B.C. Ooi (2012). LogBase. Proceedings of the VLDB Endowment, vol. 5, no. 10, pp. 1004-1015. https://doi.org/10.14778/2336664.2336673

[68] C. Riegger, I. Petrov (2024). Storage Management with Multi-Version Partitioned BTrees. Information Systems, vol. 125, pp. 102403. https://doi.org/10.1016/j.is.2024.102403

APPENDIX A: REPRODUCIBILITY

The Revon GitHub repository provides the code and experimental records accompanying this paper. The Linux reproducibility archive and the Windows reproducibility archive are in its paper/ directory. README files explain dependency setup, public-data acquisition, experiment execution and evidence verification [1]. Every bundle retains the executed sources, fixed protocol, raw measurements and internal audits.

The primary Linux, supplementary Linux and Windows campaigns have different protocols and must be analysed separately. Documented preparation steps rebuild TLC inputs and generated repositories. Public-source metadata records licences, acquisition versions, checksums and the CO2 exclusion applied before timing. Use disposable copies with localized data paths when running auditors so the collected records are preserved. Semantic-hash matches establish consistency, but neither reproduce elapsed times nor guarantee identical timings on a different machine.

APPENDIX B: METHODS AND EVIDENCE DETAILS

Linux bundles retain the executed sources and hashes, environments, fixed workloads, execution order, raw observations and audits. The workers use PYTHONHASHSEED=20261008. Preflight passes 67 unit tests for the primary campaign and 69 for the supplement before measurement begins. Supplement preflight detects an incomplete transfer lacking the test_data directory. That empty fixture directory is restored and the complete suite passes before timing starts. The setup failure is kept in the execution record.

Each primary execution constructs a repository in a new worker. Deterministic shuffling and rotation set model order within repetition blocks. Import timing stops at the first durable version. The median of individual commit timers gives incremental-commit latency for a history. Diff compares the first and last versions, emitting sorted changed keys with old/new-value hashes as canonical JSON. Checkout writes the entire sorted key/value state as canonical UTF-8 JSON. Revon is called through an in-process API; Dolt timing includes CLI launch, JSON parsing and output normalization. Workflow ratios consequently cannot be read as engine-only speed comparisons.

Correctness checks are excluded from operation timers. For each primary execution, we verify the initial and final state hashes, all H+1 historical states, and the observed canonical diff against a separately built workload oracle. Verification records contain expected and observed hashes. Audits regenerate workloads, check matrix membership, selected paths

and source/data/protocol identities, and recompute summaries. Downloaded records are audited locally too. The checks establish internal integrity, not independent repetition of the timing experiment.

We report medians and interquartile ranges (IQRs) for the seven measured repository trials after excluding warm-ups. Latency ratios use matched trial pairs: we compute the median ratio and a paired percentile-bootstrap interval with 20,000 resamples. Values above one favour the denominator. The exploratory 95% intervals apply to repeated timing of a fixed workload on one VM and have no population or multiple-comparison interpretation. A median of paired ratios may differ from a ratio of latency medians. We keep every completed trial, including timing outliers.

A parent process samples RSS, CPU and I/O for the worker and its descendants every 10 ms. Peaks include storage work as well as workload generation, oracle state and verification, so the storage engine cannot be assigned the whole peak. Short-lived subprocesses or brief peaks may occur between samples. The primary Linux sampler treats an all-zero read counter as missing, leaving its records unable to support a read-I/O conclusion. The supplementary campaign distinguishes measured zero from unavailable counters and is analysed separately. Cached data can be examined logically while physical read bytes are recorded as zero.

The Linux footprint measurement is directory size before closing or compaction. It includes indexes, metadata and representation costs. Logical-payload normalization and selected first-trial compaction probes are provided as well. These measurements concern the complete workflow rather than tree overhead alone. CPU, RSS and I/O cover the full worker lifecycle, a broader interval than the individual operation timers.

In the auxiliary Windows threshold experiment, calibration and validation profiles are predefined and non-overlapping. Candidates are 1,024, 2,048, 4,096, 8,192 and 16,384 operations. Minimizing normalized calibration latency selects T=16,384 before validation, giving a held-out normalized score of 1.032 against 1.297 for T=4,096. Always-log scores 1.000 in both splits. The cached-query observations belong to that experiment's recorded source and environment and remain separate from Linux timings. A largest-candidate winner does not establish an optimum. Long-history and three-seed pilots supply exploratory context.

The Linux threshold control uses a fixed T=16,384 with 10,000 rows and 512 changes per commit. Intervals H=16 and H=64 accumulate 8,192 and 32,768 operations. Forced log, forced Merkle and hybrid run in separate workers, each with two warm-ups and seven measured trials. The threshold is not chosen from Linux results. These cases test transfer of the auxiliary calibration choice, without another calibration/validation cycle.

For Linux geometry we fix 10,000 rows, ten commits, 100 changes per commit, 32-byte synthetic values and seed 20260824. Merkle diff is forced for b4-d6, b8-d3, b8-d4, b8-d5 and b16-d3. Every configuration has fresh workers for two warm-ups and seven measured trials, with order rotated to limit time/configuration confounding. Each execution checks eleven historical states and the endpoint diff outside timing. The output contract matches the workflow experiment. Geometry-adapter peaks from its in-process monitor are kept separate from external RSS measurements.

The Windows protocol and case manifest specify calibration seeds 202610161/202610162 and disjoint validation/sensitivity seeds 202610171-202610173. Workers use PYTHONHASHSEED=20261016. Monitoring uses seed 202610190, generated public corrections use 202610201-202610203, and scale uses 202610221-202610223. Each worker builds fresh persistent state, but its queries share one repository.

We timestamp the threshold before validation and do not repeat selection afterwards. Linux/Windows rows are analysed separately.

Windows instrumentation invokes the native Dolt JSON-diff command under the adapter's parser contract and records explicit phase timers. Preflight compares observed output with the native adapter and the independent oracle. Revon checks observed output hashes, every historical-state hash, reverse/identity requests, object integrity and reopening outside timing. Setup checks and data-quality exclusions happen before measurement and are documented. The retained query matrix and timing-source hashes specify the execution conditions.

The portable audit uses a disposable copy of the evidence. In a fresh interpreter it imports the executed source and localizes the public-data path from the records. It regenerates workloads and state/diff oracles, checks every configuration, trial and order identity, and recomputes numerical summaries. Collected-file hashes are then verified. The archives include these checks and data-acquisition recipes for consistency verification; reproducing timing requires running the actual experiments.

Table 16
Checksums of public data and the Linux protocol

Item	Definition
Source	NYC TLC yellow_tripdata_2024-01.parquet; 2,964,624 source rows
Raw SHA-256	c4d59da7bbc8abaeeeb1727947ee93d9891a71acb42854bd80db1571b2030510
Prepared million-row SHA-256	6608b7680e82432ca22f90a71d4994676ffd290c4fc85f17f91fab03d4c7b661
P1	100,000 records; H=10; C=100; 2 warm-ups + 7 measured trials per system
Scale diagnostic	1,000,000 records; H=10; C=1,000; one trial per system; no interval or ranking

Table 17
Windows execution matrix; cached queries are counted separately from independent history or repository repetitions

Component	Fresh workers	Histories / control	Warm-up queries	Measured queries
Calibration	12	2 seeds × 2 localities × 3 densities	144	504
Held-out	18	3 different seeds × same profiles	216	756
Sensitivity	21	3 seeds × 7 matched variations	252	882
Monitor diagnostic	6	3 off/on pairs, one fixed history	72	252
Public sources	18	2 sources × 3 runs × M/H/Dolt	36	126
Scale	18	2 sizes × 3 seeds × M/H/Dolt	36	126
Total	93	Different campaigns are never pooled	756	2646